

EE708 – Fundamental of Data
Science and Machine Intelligence

Digit Recognition

Presented By - Group 22

Peeyush Sahu (230750)

Yash Kumar (231187)

Dhruv Bajaj (230365)

Bipin Kumar (230300)

Anshika Singh (230163)

Submission Date

11th April 2026

Course Instructor

Prof. Rajesh M Hegde



Problem:

Digit Recognition from Short Audio Clips

Goal : Build a model that can recognize spoken digits from **0 to 9** .

Input : **short .wav** audio clips of spoken digits.

Output : predicted digit label for each test audio clip .

Challenge : test data contains **unseen speakers** and **noisy / augmented recordings**.

Main objective : learn digit-relevant acoustic patterns that generalize well across speakers and recording conditions.

Evaluation metric : **Macro F1-score** (averaged over 10 classes).

What the model has to do:

1. Training data is provided with labels in `train.csv`
2. Test audio files are provided separately, but **labels are hidden**.
3. Each row in the CSV contains an id that maps directly to one audio file .
4. **Training path** : `train_audio/{id}.wav`
5. **Test path** : `test_audio/{id}.wav`
6. Final submission must predict a label in `{0,1,2,3,4,5,6,7,8,9}` for every test id.

Dataset Understanding

Dataset structure

- **Training set:**
 - o train_audio/ contains labeled .wav clips
 - o train.csv provides the corresponding digit labels
- **Test set:**
 - o test_audio/ contains unlabeled .wav clips
 - o test.csv / sample submission provides the ids only
- All files are in .wav format .
- Each audio file is linked to a unique id without the .wav extension.

Important mapping:

- train_audio/{id}.wav
- test_audio/{id}.wav

This means the entire workflow depends on matching the CSV ids correctly with the audio files.

EDA (Exploratory Data Analysis)

What matters most in this dataset

- The task is a **10-class classification problem**.
- Audio clips are short, so the important information is mainly in the **time-frequency pattern**.
- Since the test set contains **unseen speakers**, speaker-specific cues are unreliable.
- Because recordings may be noisy or augmented, preprocessing must make the model robust.
- The dataset is best treated as an **audio classification** problem rather than plain sequence classification.

Practical observation:

- A good solution should not depend on raw waveform memorization.
- Converting audio into a structured representation like a **spectrogram** is a better fit.

Approach

1) Preprocessing

Audio Processing Pipeline

- Audio is loaded in mono and resampled to **16 kHz**
- Each clip is fixed to **1 second** by padding or center trimming
- Audio is converted into a **Mel-spectrogram**
- The spectrogram is normalized to improve stability during training
- To enrich the representation, two extra channels are added:
 - first-order delta
 - second-order delta

2) Feature Extraction

- **Fixed-length audio:** Pad/trim to exactly **1 second** (16,000 samples).
- **Mel-spectrogram:**
 - $n_{\text{mels}} = 128$, $n_{\text{fft}} = 512$, $\text{hop_length} = 160$
 - Frequency range: 50–8000 Hz

Final input format

- **3-channel** spectrogram tensor:
 - log-Mel spectrogram
 - delta
 - delta-delta
- This gives the model both static and temporal speech information.



Approach

2) Feature Extraction

1. **Log scaling** : Power spectrograms are converted to decibel (dB) scale for better dynamic range representation.
2. **Delta features** : `librosa.feature.delta` (order 1 and 2) → stack with base log-mel → 3 input channels.
3. **Precomputation** : All train/test specs saved as .npy files → avoids repeated librosa calls (saves ~5 hours).

3) Model Architecture

- **Backbone : ResNet-18** (pre-trained weights not used – train from scratch).
- **Modifications :**
 - The first convolutional layer (`conv1`) is adapted to accept the custom 3-channel Mel spectrogram inputs.
 - Removed `maxpool` (`nn.Identity()`).
 - Replaced final FC layer with:
Dropout(0.2) → Linear(512 → 256) → ReLU → Dropout(0.1) → Linear(256 → 10)
- **Total parameters** : ~11.2 million.
- **Output** : 10-class logits for digits 0–9.

Approach

4) Data Augmentation (Building Robustness Against Noise)

- **Waveform-level** (applied before spectrogram):
 - Additive white noise (**probability 0.5, amplitude 0.002–0.008**).
 - Random time shift (**circular shift up to ± 0.1 sec**).
- **Spectrogram-level** : Frequency and time masking are applied randomly to the spectrograms, forcing the model to learn from incomplete acoustic patterns.
 - Frequency masking: **2 masks**, each up to 16 bands.
 - Time masking: **2 masks**, each up to 30 frames.
- **Mixup** - Blends pairs of spectrograms and their labels during training ,smoothing decision boundaries and acting as a strong regularizer with (probability 0.5, alpha=0.4).
- **Label smoothing** = 0.1 in cross-entropy loss.

All augmentations help generalize to unseen speakers & noise.

Experiments

1) Training Setup and Validation

- **3-fold Stratified K-Fold** (keeps class distribution balanced in every fold).
- **Optimizer:** AdamW (Learning rate (lr) = $3e-4$, weight_decay = $1e-4$).
- **Scheduler:** CosineAnnealingLR ($T_{max} = 20$, eta_min = $1e-6$).
- **Batch size:** **256** (fits T4 GPU).
- **Epochs:** **35** per fold.
- **Mixed precision** (torch.cuda.amp) for speed & memory.
- **Loss:** Cross-entropy with label smoothing (0.1) + Mixup loss.
- **Checkpoint:** Save best model per fold (highest validation accuracy).
- **Validation:** After each epoch → compute loss & accuracy.
- **OOF (Out-of-Fold) predictions:** Aggregated across folds → final OOF accuracy.
- **Test-time augmentation (TTA):**
 - 5 rounds of random augmentation on test set.
 - Average softmax probabilities across rounds.
- **Ensemble:** Average predictions from all 5 folds.
- **Precomputation benefit:**
 - Training dataloader loads .npy files directly → no librosa during training.
 - Speeds up each epoch significantly.

Results

Key training metrics:

- Convergence was fast — validation accuracy crossed 99% by epoch 8 in most folds
- Training loss reduced steadily from $\sim 1.54 \rightarrow \sim 0.69$ across 35 epochs
- Best checkpoint per fold saved and ensembled for final test predictions
- TTA (5 rounds) + 5-fold ensemble used for final submission

Final submission: submission.csv generated with predicted labels for all 16,200 test samples

Best Validation Accuracy	
Fold	
Fold 1	99.55%
Fold 2	99.42%
Fold 3	99.56%
Fold 4	99.63%
Fold 5	99.54%
OOF (Overall)	99.54%

Key Takeaways

What worked well:

- 3-channel spectrogram (log-Mel + Δ + $\Delta\Delta$) gave the model both static frequency patterns and temporal speech dynamics — crucial for generalizing across speakers
- Aggressive augmentation (waveform noise, time shift, frequency/time masking, Mixup) was the single biggest factor in achieving robustness to unseen speakers and noisy recordings
- Precomputing spectrograms as .npy files eliminated repeated librosa calls and drastically reduced per-epoch time (estimated ~5 hours saved)
- ResNet-18 trained from scratch proved sufficient — a well-regularized shallow model outperformed fine-tuned pretrained networks for this narrow domain task
- Mixed precision training (torch.cuda.amp) maximized GPU utilization on T4 within Kaggle's time budget

Lessons learned:

- Speaker-independent generalization is better achieved through data augmentation than through model complexity
- Macro F1 on a well-balanced 10-class problem tracks closely with accuracy — class imbalance was not a bottleneck
- TTA and fold-ensemble together provide meaningful probability calibration, especially on edge cases
- Label smoothing (0.1) + Mixup together acted as complementary regularizers — neither alone was as effective

If we had more time / compute:

- Explore EfficientNet or lightweight transformer backbones (e.g., AST)
- Add SpecAugment with more aggressive masking
- Experiment with larger Mel filter banks ($n_{\text{mels}} = 256$)



x x x x

Thank You

For Your Attention

